Generative AI & LLM – Assignment 2 (IML Lab)

Report and Solutions

Faiq Imran
Roll No.: **MSDS25007**
Information Technology University (ITU)

Contents

1 Convolutional Neural Networks **1**

2 Encoder-only Transformers **2**

3 Generative Adversarial Network (Inpainting) **3**

4 Variational Autoencoders **4**

5 Understanding Diffusion Models **4**

6 Experiments: DDPM Training on Cat Images **6**

6.1 Setup 6
6.2 Training loss 6
6.3 Generated samples 6
6.4 Forward diffusion visualization 8
6.5 Backward diffusion visualization 9
6.6 Overall observations 9

1 Convolutional Neural Networks

1.1.a Padding amounts

For the kernel of size $K_h \times K_w$, the row offset of the input pixel read for output row h is $m - h = -\lfloor K_h/2 \rfloor + (i-1)$ with $i \in \{1, \ldots, K_h\}$, so it ranges from $-\lfloor K_h/2 \rfloor$ to $K_h - 1 - \lfloor K_h/2 \rfloor$. The same holds for columns. The kernel therefore reaches this far past each border:

Answer.

$$p_t = \left\lfloor \frac{K_h}{2} \right\rfloor, \quad p_b = \left\lfloor \frac{K_h - 1}{2} \right\rfloor, \quad p_l = \left\lfloor \frac{K_w}{2} \right\rfloor, \quad p_r = \left\lfloor \frac{K_w - 1}{2} \right\rfloor.$$

(For odd K these are all equal to $\lfloor K/2 \rfloor$; for even K the bottom/right padding is one less than top/left.)

1.1.b Convolution on the padded input

After zero-padding, $x' \in \mathbb{R}^{C \times (N_h + p_t + p_b) \times (N_w + p_l + p_r)}$ and original pixel (m, n) sits at $(m + p_t,\, n + p_l)$ in x'. Substituting $m = h - p_t + (i-1)$ and $n = w - p_l + (j-1)$ gives index $h + i - 1$ and $w + j - 1$, which always lie inside x':

Answer.

$$y'_{1,h,w} = \theta_0 + \sum_{c=1}^{C} \sum_{i=1}^{K_h} \sum_{j=1}^{K_w} \theta_{c,i,j}\; x'_{c,\; h+i-1,\; w+j-1}, \qquad \forall h \in \{1..N_h\},\; w \in \{1..N_w\}.$$

This equals y with hallucinated (zero) pixels, since the padding cells are exactly those zeros.

1.1.c.i Role of skip connections in U-Net

Answer. Skip connections copy high-resolution feature maps from each encoder level to the matching decoder level (by concatenation). Downsampling loses fine spatial detail (edges, boundaries); the skips give that detail back to the decoder, so predictions are *precisely localised* (important for pixel-wise segmentation) while the deep path supplies the semantic context. They also give gradients a short path, which eases training.

1.1.c.ii Spatial resolution in encoder / decoder

Answer. Encoder: each stage applies convolutions then downsampling (pooling or strided conv), so height and width shrink (e.g. halve) while the number of channels grows. **Decoder:** each stage upsamples (transposed conv/interpolation), so height and width grow back to the input size while the channels shrink.

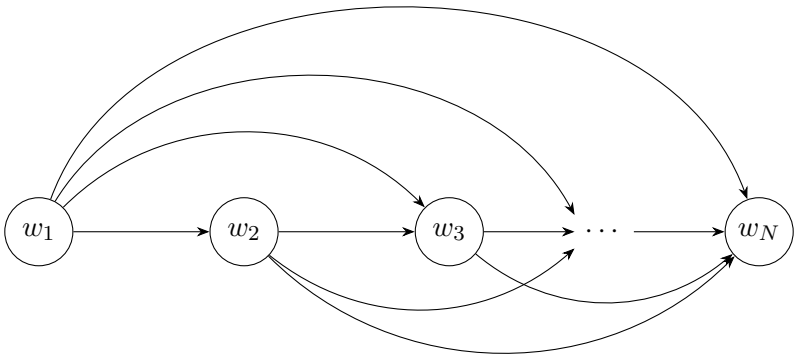
2 Encoder-only Transformers

2.1 Graphical model of a decoder-only Transformer LM

A decoder-only model uses causal masking, so each word depends on *all* earlier words:

$$p(w_1, \ldots, w_N) = \prod_{n=1}^{N} p(w_n \mid w_1, \ldots, w_{n-1}).$$

The directed graph therefore has one node per word and an edge $w_i \to w_j$ for every $i < j$ (a fully connected DAG; edges to w_N come from every earlier node).



("$\cdots$" stands for the omitted nodes $w_4, \ldots, w_{N-1}$ and their incoming edges.)

2.2 Encoder-only vs. decoder-only distributions

Answer.

- **Encoder-only** (bidirectional attention, e.g. BERT). Given the whole sentence, $s_n = \exp(Wh_n + b)$ normalised over the V tags gives a *conditional* distribution per word:

$$p(t_n = v \mid w_1, \ldots, w_N) = \frac{s_{n,v}}{\sum_{v'} s_{n,v'}}.$$

  Each position uses both left and right context, and given the input the tags are modelled as independent. It does *not* define a joint distribution $p(w_1, \ldots, w_N)$ over text, so it cannot generate sentences.

- **Decoder-only** (causal attention, e.g. GPT). Defines a full joint distribution over sequences via the chain rule above, each word conditioned only on the past, so it can sample new text.

- **Applications.** Encoder-only: understanding tasks, e.g. POS tagging, NER, classification, sentence embeddings, extractive QA. Decoder-only: generation tasks, e.g. text completion, chatbots, translation or summarisation by generation, code generation.

3 Generative Adversarial Network (Inpainting)

3.1.a Blank (a): model output

The model sees the masked image and the mask, and the final image keeps the original pixels where $m = 0$ and uses the model's fill where $m = 1$:

Answer.

$$x' = (1 - m) \odot x + m \odot g_\theta\big((1 - m) \odot x,\ m\big).$$

3.1.b Blank (b): MSE loss

Only masked pixels (those $m_{ij} = 1$) should contribute, so weight the squared error by m and normalise by the number of masked pixels:

Answer.

$$l_{\text{mse}}(\theta) = \frac{1}{\sum_{i,j} m_{ij}} \sum_{i=1}^{N_h} \sum_{j=1}^{N_w} m_{ij} \Big(x_{ij} - \big[g_\theta((1 - m) \odot x, m) \big]_{ij} \Big)^2.$$

3.1.c Blank (c): GAN-style loss

With $d_\phi(\cdot)$ the probability that an image is real and x' from 3.1.a:

Answer.

$$l_{\text{gan}}(\phi, \theta) = \log d_\phi(x) + \log\big(1 - d_\phi(x')\big),$$

where the discriminator ascends this in ϕ and the generator g_θ descends it in θ (minimax game $\min_\theta \max_\phi$). In practice one may use the non-saturating form $-\log d_\phi(x')$ for θ and add $\lambda\, l_{\text{mse}}$ for the combined objective.

3.1.d Disadvantage of MSE only

Answer. Training with MSE alone makes the model predict the *average* of all plausible fills, which gives **blurry, over-smoothed** inpainted regions that lack realistic texture and sharp detail. The adversarial loss pushes the output to look like a real image, giving sharper, more realistic results.

4 Variational Autoencoders

4.1 Gradients with and without reparameterization

Answer.

(a) **Black-box sampling:** z is treated as a constant input to the decoder, so $\hat{R} = \log p_\theta(x \mid z)$ only depends on the decoder parameters. Gradients reach only $\boldsymbol{w_3}, \boldsymbol{b_3}$. The encoder parameters w_1, b_1, w_2, b_2 receive *no* gradient.

(b) **Reparameterization:** $z = \mu_\phi(x) + \sigma_\phi(x)\,\varepsilon$ with $\sigma_\phi = \exp\big(\frac{1}{2}(w_2 x + b_2)\big)$ is a differentiable function of the encoder parameters (the randomness is moved to ε). Gradients reach **all six**: $w_1, b_1, w_2, b_2, w_3, b_3$.

4.2 Batch loss for the video VAE

Per sample: MSE normalised by the $3FHW$ values, plus β times the KL to the prior $\mathcal{N}(0,1)$. With $\mu_0 = \mu_\phi(x_i)$, $\sigma_0^2 = \sigma_\phi^2(x_i)$, $\mu_1 = 0$, $\sigma_1^2 = 1$ the hint gives $D_{KL} = \frac{1}{2}\big(-\log\sigma_\phi^2(x_i) + \sigma_\phi^2(x_i) + \mu_\phi(x_i)^2 - 1\big)$. Averaging over the batch of N:

Answer.

$$\boxed{\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{3FHW}\big\|x_i - \tilde{x}_i\big\|_2^2 + \frac{\beta}{2}\Big(\sigma_\phi^2(x_i) + \mu_\phi(x_i)^2 - 1 - \log\sigma_\phi^2(x_i)\Big)\right]}$$

4.3 Effect of increasing β

Answer. "The distribution $q_\phi(z \mid x)$ approaches $\mathcal{N}(0, I)$ but reconstructions become worse." A larger β weights the KL term more, pulling the posterior toward the prior; the latent then carries less information about x, so reconstruction quality drops.

5 Understanding Diffusion Models

5.1 ELBO surgery (proof)

Start from Eq. (7), expand $p_\theta(x_{0:T}) = p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$ and $q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$:

$$\text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)}\Big[\log p(x_T) + \sum_{t=1}^{T}\log p_\theta(x_{t-1} \mid x_t) - \sum_{t=1}^{T}\log q(x_t \mid x_{t-1})\Big].$$

Step 1 (split off $t = 1$ of the reverse model and re-index the rest). $\sum_{t=1}^{T}\log p_\theta(x_{t-1}|x_t) = \log p_\theta(x_0|x_1) + \sum_{t=1}^{T-1}\log p_\theta(x_t|x_{t+1})$.

Step 2 (split off $t = T$ of the forward process). $\sum_{t=1}^{T}\log q(x_t|x_{t-1}) = \sum_{t=1}^{T-1}\log q(x_t|x_{t-1}) + \log q(x_T|x_{T-1})$.

Step 3 (combine).

$$\text{ELBO} = \mathbb{E}_q\big[\log p_\theta(x_0|x_1)\big] - \sum_{t=1}^{T-1}\mathbb{E}_q\Big[\log\frac{q(x_t|x_{t-1})}{p_\theta(x_t|x_{t+1})}\Big] + \underbrace{\mathbb{E}_q\big[\log p(x_T) - \log q(x_T|x_{T-1})\big]}_{C}.$$

C contains only $p(x_T) = \mathcal{N}(0, I)$ (fixed, no θ) and the fixed forward process, so it is constant in θ.

Step 4 (reduce expectations). Each remaining term depends on only a few variables, so the others marginalise out ($\mathbb{E}_{q(a,b,c)}[f(b,c)] = \mathbb{E}_{q(b,c)}[f(b,c)]$). By the Markov property the term for t involves only (x_{t-1}, x_t, x_{t+1}), and the first term only x_1:

$$\log p(x) \geq \mathbb{E}_{q(x_1|x_0)}[\log p_\theta(x_0|x_1)] - \sum_{t=1}^{T-1} \underbrace{\mathbb{E}_{q(x_{t-1},x_t,x_{t+1}|x_0)}\left[\log \frac{q(x_t|x_{t-1})}{p_\theta(x_t|x_{t+1})}\right]}_{L_t} + C. \qquad \blacksquare$$

5.2 Effect of L_t

Answer. The ELBO contains $-\sum_t L_t$, so maximizing the ELBO means *minimizing* each L_t. Since $\mathbb{E}[\log q(x_t|x_{t-1})]$ does not depend on θ, this is the same as maximizing $\mathbb{E}[\log p_\theta(x_t \mid x_{t+1})]$ over x_t drawn from the forward process. This forces the reverse model $p_\theta(x_t|x_{t+1})$ to put high probability on the states the forward process actually visits, i.e. to **learn to undo one noising step** (denoise). The contribution $-L_t$ is largest (and L_t smallest) when $p_\theta(x_t|x_{t+1})$ matches the corresponding true forward-process conditional exactly; L_t is large (bad) when p_θ gives low probability to real forward samples.

5.3 Sampling the diffusion process at $t = \tau$ (step by step)

Answer. Take x_0 from the data, then for $t = 1, \ldots, \tau$ sample $\varepsilon_t \sim \mathcal{N}(0, I)$ and set $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_t$. The result x_τ is the sample. Time complexity: $\mathcal{O}(\tau)$ (one noise draw and update per step).

5.4 Closed form $q(x_t \mid x_0)$

Using Eq. (10) on Eq. (9) with $\varepsilon_{t-1}, \varepsilon_{t-2} \sim \mathcal{N}(0, I)$ independent:

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\ &= \sqrt{\alpha_t}\big(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2}\big) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\ &= \sqrt{\alpha_t\alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}}_{\text{sum of independent Gaussians}}. \end{aligned}$$

The noise terms add in variance: $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$, so they merge into one $\sqrt{1-\alpha_t\alpha_{t-1}}\, \varepsilon$. Repeating this down to x_0 (induction on t) gives

$$x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\, \varepsilon, \;\; \varepsilon \sim \mathcal{N}(0, I) \implies q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar\alpha_t}\, x_0, (1-\bar\alpha_t) I\big), \;\; \bar\alpha_t = \prod_{i=1}^{t} \alpha_i. \qquad \blacksquare$$

5.5 One-shot sampling at $t = \tau$

Answer. Sample $\varepsilon \sim \mathcal{N}(0, I)$ once and compute $x_\tau = \sqrt{\bar\alpha_\tau}\, x_0 + \sqrt{1-\bar\alpha_\tau}\, \varepsilon$. With $\bar\alpha_\tau$ precomputed (a cumulative product done once for all t), the cost is $\mathcal{O}(1)$ in τ, versus $\mathcal{O}(\tau)$ for the step-by-step method in 5.3. This is what makes diffusion training efficient: a random t can be noised in a single step.

6 Experiments: DDPM Training on Cat Images

6.1 Setup

A DDPM was implemented with a U-Net that predicts the noise added in the forward process, and trained on a subset of the AFHQ cat images. The cosine variance schedule (offset $s = 0.008$) was used with $T \approx 50$ diffusion timesteps. The model was first trained for 1,000 steps to check the implementation, then for the full 10,000 steps.

Table 1: Training configuration of the two runs.

Setting	**Short run (6.1)**	**Full run (6.3)**
Model	U-Net (noise prediction)	U-Net (noise prediction)
Loss	L1	L1
Noise schedule	Cosine, $s = 0.008$	Cosine, $s = 0.008$
Training steps	1,000	10,000
Save & sample every	100 steps	1,000 steps
Learning rate	1×10^{-3}	1×10^{-3}
Batch size	32	32
FID calculation	Disabled	Disabled
Dataset	AFHQ cat subset	AFHQ cat subset
Expected runtime	–	$\approx$ 40 min (Colab T4 GPU)

6.2 Training loss

Table 2: Training loss (L1) at the end of each run.

Run	**Start of training**	**End of training**
1,000 steps	≈ 1.5	0.166
10,000 steps	–	≈ 0.129

The loss decreased smoothly and steadily with no instability, showing that the U-Net successfully learned to predict the noise. Going from 1,000 to 10,000 steps lowered the loss further from 0.166 to about 0.129.

6.3 Generated samples

1,000-step run. Samples were blurry and lacked detail (no fur texture or eye detail), as expected for early training; a rough cat shape was starting to appear in some samples.

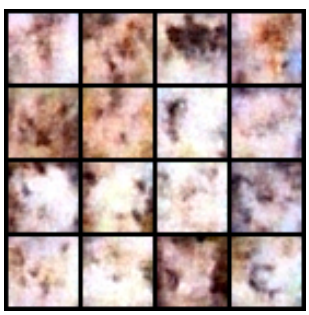


Figure 1: Samples after 1,000 training steps (16 images): blurry, with no clear detail.

10,000-step run. Samples were saved every 1,000 steps (batch of 32 images per grid). Figures 2–3 show the ten checkpoints, from step 1,000 (top left) to step 10,000 (bottom right of the second figure).



Step 1,000



Step 2,000



Step 3,000



Step 4,000

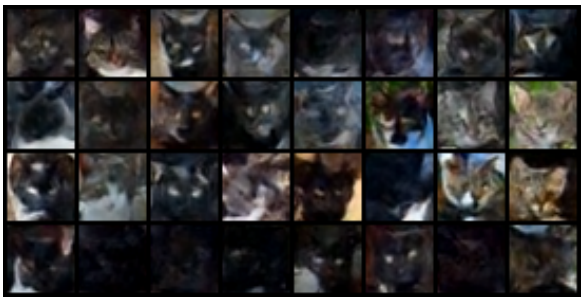


Step 5,000

Figure 2: Generated samples, checkpoints at steps 1,000 to 5,000.



Step 6,000



Step 7,000



Step 8,000



Step 9,000



Step 10,000

Figure 3: Generated samples, checkpoints at steps 6,000 to 10,000.

By 10,000 steps the cats are clearly recognisable, with distinct facial structure (eyes, nose, mouth), visible fur texture and patterns, realistic colour variation and proper ear shapes and positions.

6.4 Forward diffusion visualization

Using the trained model (10,000 steps) on the first training batch, noise was added to a real cat image at 0%, 25%, 50%, 75% and 99% of the timesteps:

Progress	**Timestep**	**Observation**
0%	$t = 0$	Original image, all details intact.
25%	$t \approx 12$–13	Slight noise; cat clearly recognisable, fur texture starts to blur.
50%	$t \approx 25$	Noticeably noisy; overall shape visible, facial features hard to distinguish.
75%	$t \approx 37$–38	Heavily corrupted; only vague outlines remain.
99%	$t \approx 49$–50	Indistinguishable from isotropic Gaussian noise.

This matches the theory: $q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ tends to pure noise as $\bar{\alpha}_t \to 0$.

6.5 Backward diffusion visualization

Starting from the fully noised image ($t \approx 49$–50), the trained U-Net denoised step by step:

Progress	Timestep	Observation
0%	$t \approx 49$–50	Pure Gaussian noise, no structure.
25%	$t \approx 37$–38	Vague shapes and colour blobs emerge.
50%	$t \approx 25$	Rough, blurry cat shape visible.
75%	$t \approx 12$–13	Cat clearly recognisable; facial features and fur start to appear.
99%	$t \approx 0$–1	Clear, high-quality cat with realistic fur, eyes and colours.

6.6 Overall observations

- **Cosine noise schedule** ($s = 0.008$) gave stable training and avoided collapse at early timesteps.
- **U-Net skip connections** preserved spatial, high-frequency detail during upsampling and were essential for fine features.
- **Training length matters**: 1,000 steps gave blurry output; 10,000 steps gave clearly recognisable cats.
- **Forward vs. backward**: the forward process destroys information with noise; the reverse process learns to rebuild it by predicting and removing that noise.
- **Limitations**: with limited compute and a subset of AFHQ, quality is below state of the art, but the experiment validates the core principles of DDPMs. FID was not computed (Section 6.2 optional).